NUMPY

What is Numpy:

Numpy : Numerical Python
It's a Python library for numerical & scientific computing
Core feature → nd array (N-dimensional array), which is faster, more efficient, and more powerful than normal Python lists.

Why use Numpy?

Speed:- written in C, so operations run much faster than lists.

Memory Efficient:- Stores data compactly

Math Ready:- Has built-in function for liner algebra, Statistics, Probability. etc.

Foundation for Data Science-> Libraries like Pandas, Tensorflow, Scikit-learn are built on top of Numpy.

Broadcasting:- Handle array arithmetic without writing loops

Import

```
Import numpy as np
```

② Numpy Arrays (ndarray)

- Core object : np.array()
- Array are homegeneous (all elements same type)

```
arr = np.array([1,2,3])      # 1D array
arrD = np.array([[1,2,],[3,4]])  # 2D array.
```

Array attributes :-

```
arr.shape    # Dimensions
arr.ndim     # no. of Dimensions
arr.size     # no. of elements
arr.dtype    # Data type
```

③ Array Creation

- From lists/tuples : np.array([1,2,3])
- Predefined arrays :
- np.zeros((3,3)) → all. zeros
- np.ones((2,2)) → all, zeros
- np.eye(3) → identity matrix
- np.full((2,3),7) ⇒ filled with 7

- Ranges
- np.arange(0,10,2) → like range()
- np.linspace(0,1,5) → equally spaced values.

④ Indexing & Slicing

- works like Python lists but more powerfull

```
arr = np.array([10, 20, 30, 40, 50])
arr[0]      # 10
arr[-1)     # 50
arr[1:4]    # [20, 30, 40]
```

Multi-dim

arr2D = np.array([[1, 2, 3], [4, 5, 6]])

arr2D [1,2] # 6

arr2D [:, 1] # column [2, 5]

5. Operations on Arrays

Arithmetic : element-wise

arr + 2, arr * 3, arr ** 2

- comparison:

np.dot(A,B) # matrix multiplication

~~np.dot(A,B)~~

A @ B # shortcut

- Matrix operations:

arr > 15 # Boolean mask

6. Broadcasting

- Automatic expansion of smaller arrays.

arr = np.array([1, 2, 3])

arr + 10 # [11, 12, 13]

7. Mathematical Functions

np.sum() np.mean() np.median() np.std() np.var()

np.min() np.max() np.argmax() np.argmin()

np.sin() np.cos() np.log() np.exp()

8. Reshaping & Stacking

- arr.reshape(2,3) → change shape.
- arr.flatten() → 1D array

Stacking

```
np.hstack([a,b])   # horizontal
np.vstack([a,b])   # vertical
```

9. Random Module

```
np.random.rand(3,3)  → random floats [0,1)
np.random.randn(3,3) → normal distribution
np.random.randint(1,10,5) → random integers
np.random.seed(42) → reproducible results
```

10. Advanced Indexing

- Boolean indexing

```
arr[arr > 10]
```

- Fancy indexing

```
arr[[0,2,4]]
```

11. Linear Algebra

```
np.linalg.inv(A)   # inverse
np.linalg.det(A)   # determinant
np.linalg.eig(A)   # eigenvalues & vectors.
```